

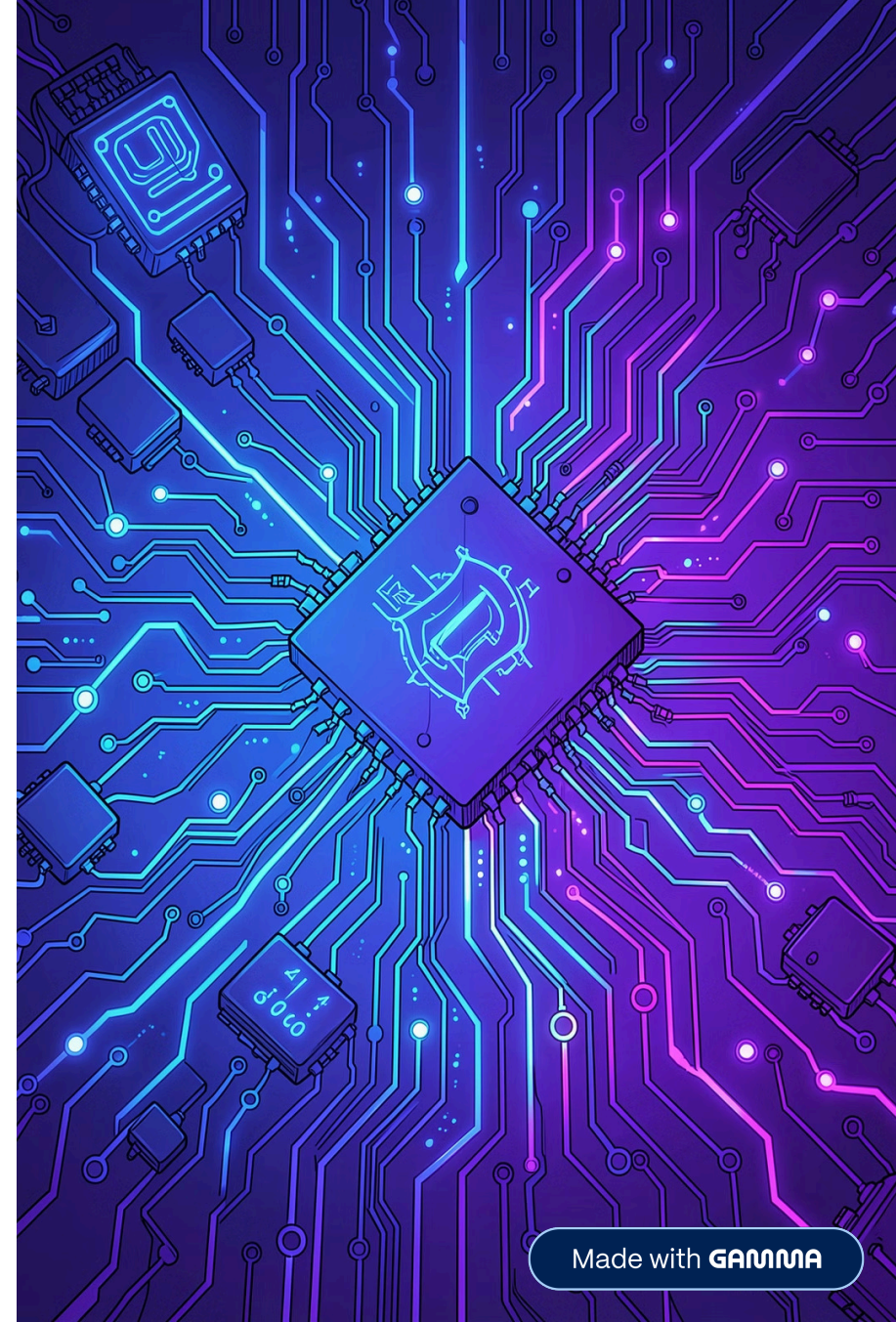
Time & Space Complexity

Big O · Ω · Θ Notations | Best / Worst / Average Case | Amortized Analysis

ALGORITHMS

DATA STRUCTURES

BEGINNER-FRIENDLY





What is Time Complexity?

Time complexity measures **how fast an algorithm runs** as the input grows. We don't count seconds — we count **how the number of steps scales** with input size.

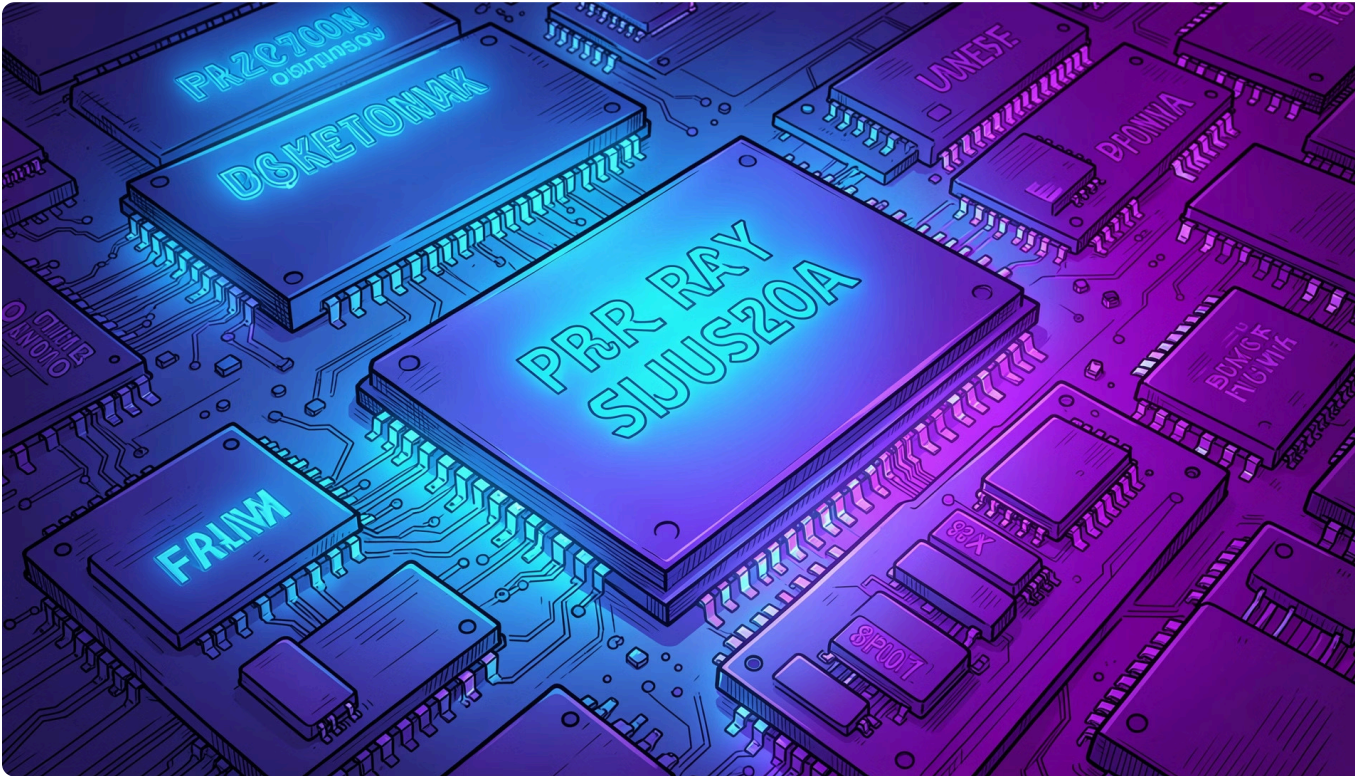
10 Items

A simple loop runs ~10 steps — nearly instant

10,000 Items

Same loop now runs ~10,000 steps — notice the growth

What is Space Complexity?



Space complexity measures **how much extra memory** an algorithm uses relative to input size.

$O(1)$ — Constant

A single variable like `sum = 0` uses fixed memory

$O(n)$ — Linear

Storing an array of `n` elements grows with input

Why Analyze Complexity?

Complexity analysis lets us **compare algorithms objectively** — before writing a single line of code.



Machine-Independent

Results don't depend on CPU speed or hardware — they're universal



Compare Algorithms

Pick the better solution before implementation, not after



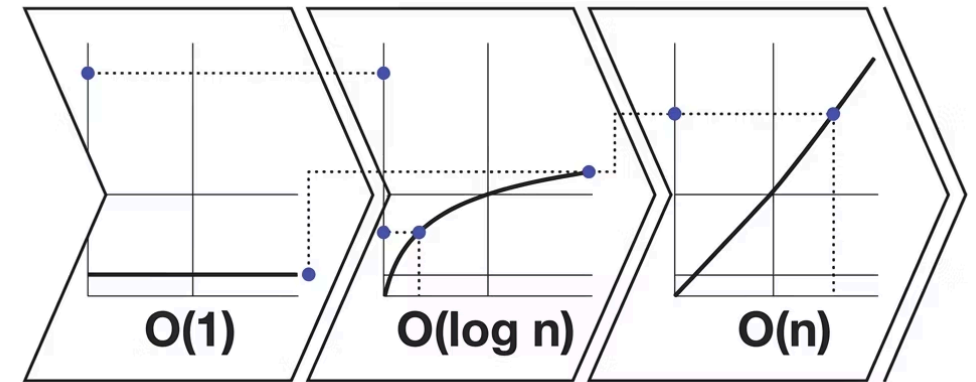
Predict Performance

Know how your code behaves at scale — 100 items or 10 million

Big O Notation — Worst Case

Big O describes the **upper bound** — the maximum steps an algorithm takes. It answers: *"What's the worst that can happen?"*

i **Example:** Searching for a value at the **last position** in an array requires checking all **n** elements → **O(n)**



Ω and Θ — Best Case & Tight Bound



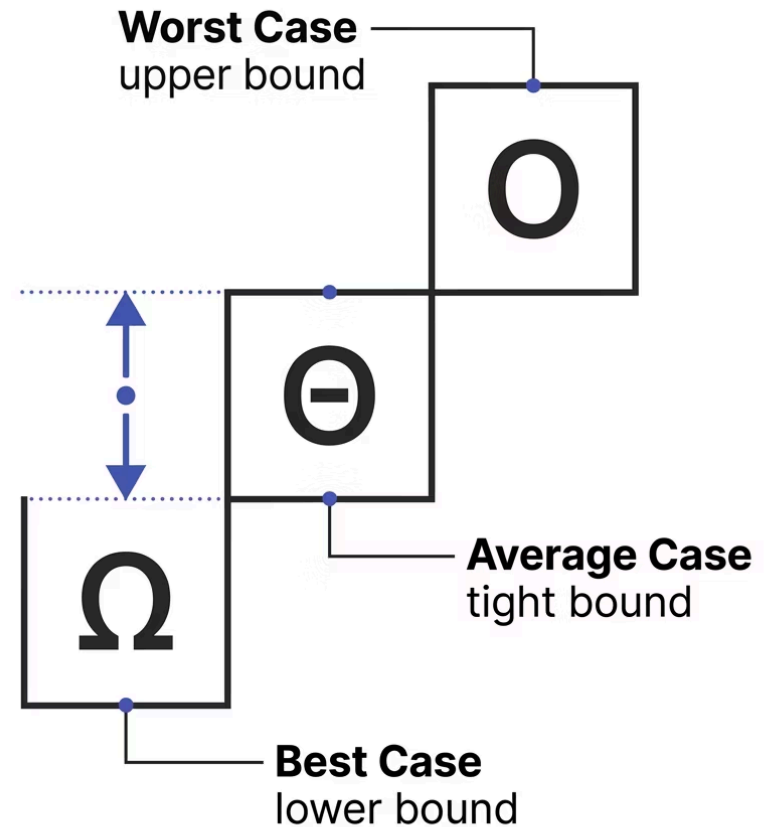
Ω — Omega (Best Case)

The **lower bound** — minimum steps needed.
Example: finding an element at the **first index**
→ $\Omega(1)$



Θ — Theta (Average Case)

The **tight bound** — when best and worst case are the same. Θ sits between O and Ω , giving a precise estimate.



Best / Worst / Average Case



Best Case — Ω

Element found at **index 0**. Only 1 comparison needed $\rightarrow \Omega(1)$



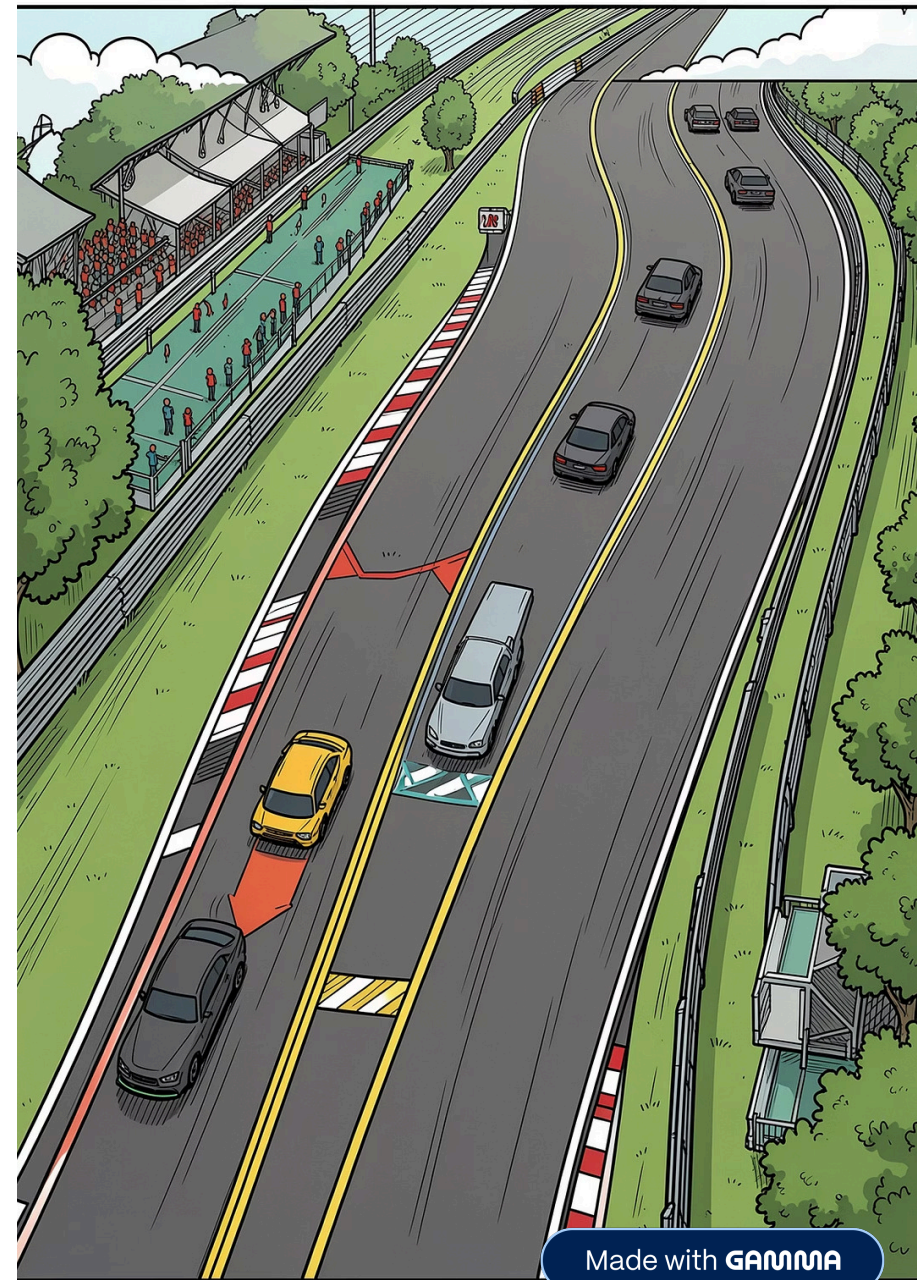
Worst Case — O

Element at **last index** or not present. All n items checked $\rightarrow O(n)$



Average Case — Θ

Element somewhere in the **middle**. About $n/2$ checks $\rightarrow$ still $\Theta(n)$



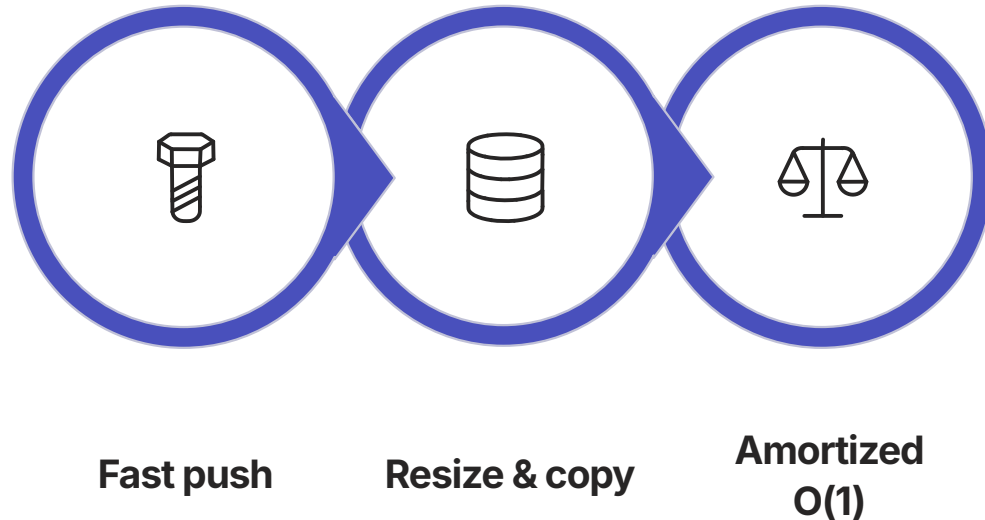
Amortized Analysis

Some operations are **slow occasionally** but **fast most of the time**. Amortized analysis spreads the rare costly operation across many cheap ones.



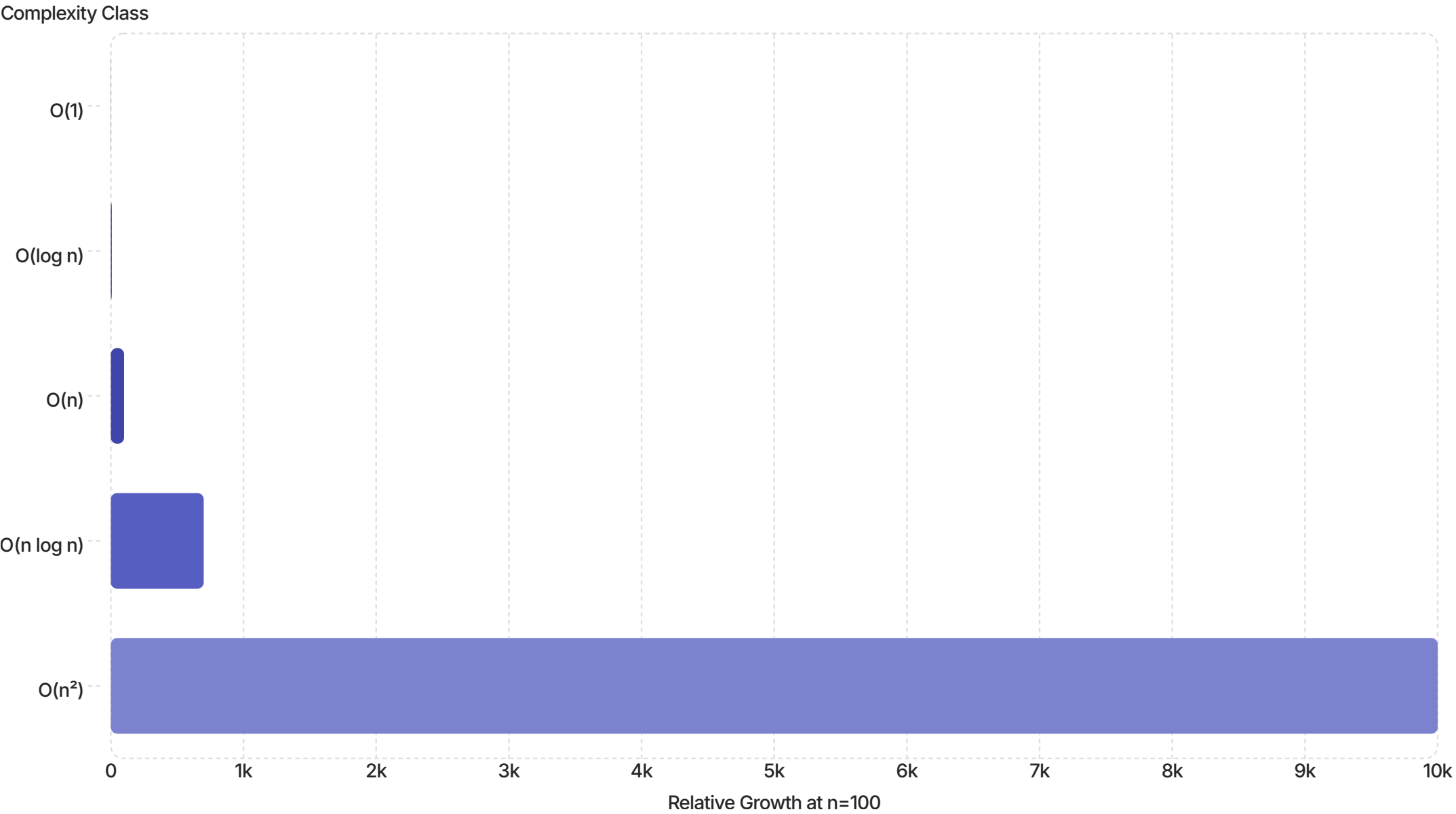
Classic Example:

`vector.push_back()` — most pushes are $O(1)$, but when the array is full it doubles in size ($O(n)$). Averaged out, each push is still **$O(1)$ amortized**.



Common Complexity Classes

Notation	Name	Example
$O(1)$	Constant	Array access by index
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Linear search, single loop
$O(n \log n)$	Linearithmic	Merge sort, quick sort
$O(n^2)$	Quadratic	Bubble sort, nested loops



Practice MCQs

Test your understanding — pick the best answer for each question.

1

Time complexity measures...

- A) Seconds of runtime B) **Growth rate of steps** C) Memory used

2

Big O represents...

- A) Best case B) Average case C) **Worst case upper bound**

3

Ω (Omega) represents...

- A) **Best case lower bound** B) Worst case C) Memory usage

4

Θ (Theta) means...

- A) Worst case B) **Tight bound (best = worst)** C) Space complexity

5

Binary search is...

- A) $O(n)$ B) **$O(\log n)$** C) $O(n^2)$

6

Bubble sort is...

- A) $O(n \log n)$ B) $O(\log n)$ C) **$O(n^2)$**

7

Amortized $O(1)$ means...

- A) Always constant B) **Average cost is constant** C) Worst case is constant

8

Space complexity of merge sort...

- A) $O(1)$ B) **$O(n)$** C) $O(\log n)$

9

Best case of linear search...

- A) $O(n)$ B) **$O(1)$** C) $O(\log n)$

10

Why analyze complexity?

- A) Count seconds B) **Compare algorithms independently** C) Measure RAM only